Where Does the Cost of Verifying AI-Generated Formal Artifacts Come From?

Anonymous research draft

Abstract

AI systems produce mathematical libraries, verified programs and streams of proof candidates. Their verification costs span library loading, elaboration, proof construction, kernel reduction and operating-system work. We characterize these costs, using the research-level `openai/math` library as the primary workload and retained proof/program candidate streams as contrasts. A complete source census covers 163,097 Lean modules; bounded native measurements retain successful, failed and dependency-blocked work. In 26 randomly selected modules compiled with standard compiler artifacts, system CPU accounts for 28.26% of measured compiler CPU. A large Grid module records 45.38 million voluntary context switches and averages 1.33 CPU cores despite eight requested workers. The default root omits 33.30% of the release's lexical `decide +kernel` occurrences. Budget-failure exposure varies across candidate streams, and profiling can change resource-bounded outcomes: seven of 200 diagnostic source pairs change error flags. We distinguish these observations from semantic falsity, whole-build costs and accelerator opportunity. [**RESULT: Public generated-software output characterization, complete cost-estimation coverage, import-only attribution and system-CPU causes.**] Certificate-repair experiments illustrate when changing proof representation can help pathological reductions and when it adds cost. The contribution is a characterization and its implications for verifier design, rather than a new branching service or certificate algorithm.

1 Introduction

AI-generated formal artifacts now range from individual proof attempts to research mathematical libraries and multi-module verified programs. Verification supplies the acceptance signal for these artifacts, but the cost of obtaining that signal is not a single solver call. Lean must prepare imported declarations, elaborate source, execute tactics, construct proof terms and check them in the kernel [3]. SMT-backed program verification adds source translation and solver interaction [6]. The cheapest stage to optimize may account for little of the complete cost.

This paper asks where that cost comes from. Our primary workload is the released `openai/math` research library [21], rather than treating short contest proofs as representative of all generated formal artifacts. We separately examine the availability of generated software repositories and use miniF2F, ProofNet and AutoVerus candidate streams as contrasts. We use existing released outputs; we do not generate a new evaluation corpus. Repository scaffolds, reference proofs and reported scores alone do not establish that evaluated model outputs are available.

Our central observation is that verification exposes different bottlenecks at different boundaries. Source-library compilation has substantial system CPU and repeated process-level preparation. Some candidate streams spend most of their measured work on requests that end at resource limits; others have a much smaller slow tail. Profiling can itself alter those limits. Even the selected build domain changes the apparent amount of computational proof work. Consequently, a lexical tactic count, a successful compiler subset or an operation-only speedup cannot stand in for verification cost.

We organize the study around four questions:

RQ1 How do artifact granularity, released-source coverage and build scope change the verification workload being measured?

RQ2 How do imports, source processing, proof construction, kernel checking and operating-system work contribute to CPU, latency and memory?

RQ3 How do failures, resource limits and observation methods change cost distributions and the reproducibility of verifier outcomes?

RQ4 Which system-design implications survive complete verification, and where do targeted proof-representation changes fail to generalize?

Each finding section presents its evidence, measurement boundary and implication. We do not assume that all verification workloads need the same cache, scheduler, proof representation or accelerator. Our contributions are a source/build-domain characterization, a boundary-aware account of measured native costs and outcomes, and case studies testing the resulting design implications. Missing results remain explicit; the current draft does not claim a complete full-library build or completed generated-software evaluation.

2 Workloads and measurement boundaries

2.1 Research mathematics is the primary workload

We pin `openai/math` to revision `4eaee875`, Lean 4.34.1 and Mathlib `d13f23b7` [21]. The source census enumerates all 163,097 Lean files and 1,785,499,363 source bytes. Comment- and string-filtered counts are lexical occurrences, not executed calls or accepted declarations. The internal import graph contains 370,864 edges, a longest unit-weight chain of 451 modules and a maximum same-depth layer of 9,225 modules. These structural statistics do not measure a timed critical path or safe runtime concurrency.

The initial probability sample draws two modules without replacement in each of 21 source-size, import-depth and kernel-token strata. Its 42 targets retain their original inclusion probabilities. Prerequisite modules and purposeful stress cases receive no additional population weight. All selected but uncompleted modules remain visible. Separate native points change compiler artifacts or resource settings; we do not pool their clocks.

At the standard-artifact point, 26 of the 42 original random targets compile without source edits or sorry warnings, completing 11 strata. Lean emits `.olean`, `.ilean` and `.c` artifacts. The measured target compiler CPU sums to 712.010 seconds. This excludes Lake driver execution, prerequisite preparation, cache acquisition, linking and unmeasured targets. An older olean-only point and its dependency continuations remain separate. For example, three completed continuations contain 69, 53 and 330 actual compiler invocations, but add only five original probability-target costs to that older frame. One purposeful algorithm target has zero sampling weight. Dependency work is real cost, not additional independent samples.

[**RESULT: Complete all-source and default-root CPU totals with uncertainty, remaining probability-target outcomes and complete build lifecycle costs.**] For a completed stratum, the estimated total is its population size times the sample mean. Its finite-population variance uses sampling without replacement. Two observations per stratum leave nominal normal-cost intervals fragile under heavy tails. Incomplete strata are unavailable, not zero. We report neither a successful-subset full-library estimate nor a finite global confidence interval.

2.2 Generated software requires released outputs

Table 1 separates publicly described tasks from usable generated artifacts. Vero describes 43 multi-module Lean 4 instances and joint code-and-proof and proof-only modes [30]. Its public repository has benchmark scaffolds and a historical BankLedger solution, but our inspected tree does not establish release of the evaluated agent-run output population. We do not treat the reference solution as an evaluated model output or run new generation.

Vericoding does release filled-in generated sources [2]. Our complete pinned tree audit identifies 626 Lean files under `vericoded`; an inspected example includes an explicit specification axiom. These are available candidates, not 626 established valid proofs. Native replay must preserve specifications, declared axioms and admissions, and distinguish individual functions from complete repository builds. [**RESULT: Measured software-output population, toolchain-qualified builds, per-stage costs and repository-level outcomes.**]

Lean Kernel Verify is a distinct software verification boundary: a frontend lowers compiler IR to KIR and Lean checks concrete-case properties using `native_decide` [19]. Its published checker can prove a safety violation; such a result is not a Lean proof failure. The public README states that some full frontend paths still require its originating monorepo. Released fixtures are useful characterization inputs, but do not by themselves establish availability of the generated software histories sought here. We keep these provenance and trust distinctions explicit.

2.3 Candidate streams are contrasts, not substitutes

We retain the original 7,808 miniF2F sources, 5,952 ProofNet candidates per recorded service pass, and 3,566 AutoVerus source candidates. AutoVerus outputs include unsuccessful and intermediate attempts from its released artifact [20]. Released EPFL evaluation and APRIL mutation/reference subsets supply additional bounded contrasts [4, 27]. These streams differ in source provenance, toolchain, selection, budgets and timing boundary. No cross-hardware normalization makes them a single population. Their verification costs exclude generation unless explicitly identified as publisher log accounting.

2.4 Measurements retain their own denominators

A compiler process, a frontend body, a warm extra declaration check and a concurrent service run are different intervals. We record user and system CPU, elapsed time, memory, configured workers, native budgets, output artifacts and terminal outcomes. Summed overlapping request intervals are not service makespan. Cumulative profiler phases can overlap and are not added into a wall-time partition. An extra warm kernel check excludes initial proof construction and imports.

Software source	Public-artifact evidence	Admission and current measurement scope
Vero	43 multi-module instances described; inspected repository `0a7325df` contains scaffolds and reference material	Evaluated agent outputs not established by this audit; native generated-output costs [**RESULT: pending output provenance**].
Vericoding	Pinned `387cd699` has a `vericoded` tree with 626 Lean, 962 Rust and 1,754 Dafny source files	Generated solution files are public; per-file model/experiment joins, admissions and native costs [**RESULT: pending**].
Lean Kernel Verify	Pinned `129e65bf` publishes Lean checking code and KIR fixtures, using `native_decide`	A released checker/fixture workload; AI-authorship/output provenance and native repository costs [**RESULT: pending**].

Table 1: Software-workload availability audit, not completed verification. No candidate is generated to fill a missing release. Public files and publisher outcomes do not establish accepted, axiom-clean proofs.

Domain	Modules	Kernel-decision tokens
All released source	163,097	1,354,379
Default-root closure	130,834	903,371
Outside default root	32,263	451,008

Table 2: Comment/string-filtered `decide +kernel` occurrences. The 33.30% omitted share is lexical, not a runtime or native-check share.

The historical service point uses four CPU leases and a 48 GiB aggregate RSS response policy. The library points use phobos-vt, CPUs 0–7 and version-specific Lean memory options. One-second tree samples can miss peaks; a Lean memory option and a sampled watchdog are not hard kernel RSS quotas. Earlier signal-limited captures use finite self-termination and preserve their interruptions. The later privileged configuration permits new profiling experiments, but does not retroactively change old measurements. [**RESULT: Qualified privileged perf captures, attribution controls and same-point repeated measurements from the current profiling campaign.**] All raw failures and censored or dependency-blocked work remain retained.

3 The default build omits a third of computational proof sites

The released library and its default build root are different workload domains. Resolving the publisher's source-directory overrides and imports, the default `OAI` root reaches 130,834 of 163,097 modules. The remaining 32,263 modules contain 387,698,422 source bytes.

All 562 PlanarPacking and 466 Transonic modules lie outside that default root at the pin. They contain 255,199 and 193,076 such occurrences. Thus a default-root build can omit precisely the computational proof families that motivate kernel-optimization studies. Conversely, explicitly building those sources is a workload extension, not evidence about the default command's cost.

Lexical density also overstates what is known about repeated kernel work. In 363 retained compiled modules, 45,661 auxiliary declarations match the decision-helper shape, with 42,610 globally distinct expression-type keys. The corresponding source-site count is 42,886: one source site can expand to several helpers, and imported/cached objects use another boundary. Lean's auxiliary-lemma cache is branch-local [13, 12]. Neither global source-text equality nor equal helper types establishes cache misses, full dependency equality or seconds saved.

System-design implication. A representative verification benchmark must specify both the released-source domain and the exact build target/artifact set. A compiler or cache evaluated only on the default root cannot claim coverage of omitted computational families. Population estimates must retain the original sampling probabilities when reporting a default-root subdomain; known out-of-domain membership is not a measured zero-cost compilation.

4 System CPU can dominate without using the requested cores

Across the 26 completed standard-artifact probability targets, measured user CPU is 510.823 seconds and system CPU is 201.187 seconds. System work is therefore 28.26% of the 712.010-second CPU sum. This is the aggregate of completed random draws, not a population-weighted full-library fraction: 16 selected draws remain incomplete at this point.

The purposeful Grid module exposes a more extreme operating point. It compiles in 634.280 elapsed seconds with 399.934 user and 440.563 system CPU seconds. Its 52.42% system share accompanies 45,375,721 voluntary and 529,643 involuntary context switches. Total CPU divided by wall time is 1.325 cores despite eight requested Lean workers. There are no observed major page faults. These counters establish low realized CPU utilization and substantial system work; they do not identify a lock, allocator, futex or scheduler as the cause.

Another large certificate module, Amplitudes, uses 228.898 user and 95.954 system CPU seconds over 180.276 elapsed seconds: 29.54% system CPU and 1.802 average cores at its separate point. Thus the inspected large modules do not all share a 39–52% range. **[RESULT: Matched certificate-module cohort and its system-share distribution, perf attribution to synchronization/allocation/I/O, and causal controls.]** The earlier one-worker profiled Grid run stops at its own 300-second bound; it cannot supply an isolated eight-versus-one-worker speedup.

System-design implication. Worker count is not available parallelism. Increasing executor threads or offloading arithmetic can leave synchronization, allocation and kernel entry costs unchanged. Attribution must measure user/system CPU together with elapsed time and actual runnable work, then test the suspected cause at matched semantics and budgets. The current counters motivate this profiling experiment; they do not yet justify a new parallel runtime or a claim that the bottleneck has been fixed.

5 Fresh module verification has a preparation floor

Each separately launched module compiler reconstructs its imported environment, even when dependency objects already exist. In the 26-module standard-artifact sample, the fastest complete compiler call takes 1.116 seconds and peaks at 2,598,928 KiB (2.48 GiB) leader RSS, for a 2,072-byte source. This is a measured whole-process floor in that sample, not an isolated Mathlib import timer or a universal minimum. The clock includes source processing and artifact emission. **[RESULT: Import-only versus empty-body controls, warm/cold page-cache conditions, allocation breakdown and matched retained-environment costs.]**

Established environment reuse provides a useful contrast [7, 22]. In one completed miniF2F resource comparison, four warm workers take 25,176.49 seconds and the prepared engine takes 20,323.01 seconds. All 7,808 final status aliases and 3,014 accepted IDs match. The 1.23882 ratio includes retry ordering, affinity, native work, fallback and retirement; it does not isolate copy-on-write or import elimination. Raw payloads differ, and a labelled two-field normalization still leaves 23 rejected-source diagnostic differences. One prepared parent serves the ordinary frontier, while 42 fresh exclusive resource retries and 18 different-import fallbacks remain charged. This service comparison does not estimate a library build's import share.

System-design implication. Small modules can be dominated by process/environment preparation even when their proof bodies are cheap. Import amortization is established engineering, not our novelty claim. It must be evaluated against a competent warm service and account for replicated resident memory, incompatible contexts, reloads, cancellation and cleanup [17]. A gain over fresh processes alone is insufficient evidence of service benefit. The current import-only campaign will determine how much of the observed floor can actually be removed without changing verification behavior.

Point	Sources	Budget sources	Cost share
miniF2F attempts	7,808	126	73.21%
ProofNet pass 1	5,952	70	11.57%
EPFL completed prefix	136	12	25.60%
EPFL training selection	12	6	55.04%
APRIL mutations	39	16	50.53%
APRIL references	39	4	18.94%

Table 3: Budget-exposed source time under each point's own measurement boundary. miniF2F includes physical retries; other rows use request clocks. Selection, environments and limits differ. No row is a sound early-rejection rule or a cross-workload speedup comparison.

6 Budget-failure exposure is workload dependent

A request that ends at a resource limit consumes verification capacity but does not establish a false proposition or an inherently impossible proof. We therefore separate budget exposure, ordinary nonzero exits and slow-tail concentration.

In the retained miniF2F engine point, 85 final timeouts and 41 final resource failures identify 126 distinct sources. Their physical attempt sums, including retries, total 38,795.133 of 52,992.124 seconds: 73.21% of worker-attempt time. This is not 73.21% of the concurrent 20,323.01-second makespan. A separate fixed 42-source exclusive rerun under the aggregate budget returns 41 watchdog resource failures and one timeout, with no recovered acceptance. It does not prove that those computations fail under every environment or budget.

In the first retained ProofNet pass, 70 of 5,952 sources with budget diagnostics consume 138.011 of 1,192.435 request seconds, or 11.57%. The other two passes have their own clocks and are not pooled. The selected EPFL and APRIL points also vary (Table 3); they are small diagnostic samples, not population estimates.

AutoVerus illustrates a different distinction. In three complete direct passes, the slowest 58 of 3,566 sources (the ceiling of 1.6%) consume 3.399%, 3.338% and 3.315% of summed external process time. These are *slow-tail shares*, not budget-failure shares. No capture timeout is reported in those passes. Meanwhile, 3,376 nonzero process exits consume about 95% of each pass's time. Ordinary rejection can be common without a con-

centrated long tail. Its first-pass external, reported frontend and SMT sums are respectively 639.238, 443.690 and 63.120 seconds. SMT is initialization plus query interaction elapsed time, not isolated Z3 CPU. Missing timing JSON or SMT fields remain unavailable; nested timers are not added twice.

For a common slow-tail definition, the slowest 96 ProofNet requests consume 33.55% in its first pass. The miniF2F retained budget-source witnesses prove a lower bound of 72.68% for its true slowest 125 source attempt sums. This miniF2F value is a subset-based lower bound, not a complete top-125 ranking. Neither number is interchangeable with the budget-share column.

System-design implication. Failure-aware scheduling must be justified for the target workload and consumer. Exact all-candidate rewards still require every result. First-success consumers may cancel peers after a checked success, but an unsolved task still requires all candidates. Historical resource failures are not safe semantic cache entries without complete state, resource and replay obligations. A workload with a 73% budget-exposed attempt share invites different experiments from one whose slowest 1.6% consumes only 3.4%. Generation cost also matters: AutoVerus publisher log labels spend 95.93% of aggregate logged span on inference, including waits/retries. The residual is unattributed, not a measured verification component, and cannot be combined with our later replay clocks.

7 Resource limits and profilers can change the observed answer

A verification verdict includes the executable environment, options, accepted context, resource policy and service interpretation. Lean recursion-depth, allocation-heartbeat and wall limits are distinct. A stored budget label is not necessarily reproduced by source text alone.

Six released EPFL exact-program controls pass under the reconstructed native context with original source, annotated source and an enlarged heartbeat setting. A recursion depth of 128 fails, whereas 256 and the default 512 pass. The original producer context is unavailable. This exposes a concrete sensitivity, not a proven publisher label bug. Replayed reference proofs, generated mutations and service responses must therefore preserve their separate provenance.

Observation can also change the work completed. A Lean 4.9.0-rc1 diagnostic executes 200 sources in control/observe pairs with the same diagnostic `maxHeartbeats=3000`. Both retain InfoTrees; observe additionally enables zero-threshold `trace.profiler`. Seven pairs change from a no-error flag to a budget error. Raw error arrays differ for 80 pairs. The control/observe body sums are 779.388 and 80.075 seconds; the shorter profiled time reflects earlier interruption, not acceleration. The complete process takes 1,170.883 seconds. A no-error frontend flag is not whole-source, axiom-clean service acceptance.

A later linked C-timing control preserves error flags, diagnostic arrays and InfoTrees on all 200 sources. Yet measured allocation-heartbeat deltas differ for 63 timing-off and 62 timing-on sources relative to stock. Its six serial processes take 3,081.953 seconds, exceeding the declared 1,800-second group limit; these are retained exploratory diagnostics. Even outcome equality does not establish exact budget accounting. Their 93.17% exclusive `eqRefl` body-CPU share cannot be transferred to the original service: one diagnostic call runs for 720.250 seconds inside that frame, while the service stops the request at 300 seconds.

System-design implication. A profile needs its own denominator and an explicit perturbation study. Hardware sampling, source tracing and extra kernel rechecks have different effects. Record changed outcomes before attributing stage cost, and compare raw diagnostics as well as acceptance sets. [**RESULT: Same-input privileged profiling repetitions, perturbation uncertainty and reproducibility across the generated-software workloads.**] Do not infer an accelerator ceiling or semantic equivalence by borrowing an unprofiled run's denominator.

8 Certificate repair helps selected pathological reductions

This case study tests an implication of the profiles: representation can matter more than performing the same arithmetic faster. It is not the paper's main contribution or a generic verified extraction compiler. All proposed replacements still require native checking.

A small finite-count proof groups 10,000 tests into 100 blocks and proves correspondence to the original predicate/cardinality. Its computation theorem has no axioms; the bridge has only standard logical axioms, without `sorryAx` or a native-evaluation axiom. A complete successful control takes 30.581 seconds, including a warm extra declaration check of 8.557 seconds. That subclock is not added again or compared with a separately timed untrusted numeric loop as a checking speedup.

The broader miniF2F test makes the coverage boundary explicit. For 64 exact sources in two selected pathological computation families, original execution has zero acceptances, 13 ordinary errors and 51 budget outcomes; the repair has 31 acceptances, 29 errors and four budget outcomes. A third, 32-source power family adds only two acceptances; 29 remain errors and one remains budget-limited. These are changed-source proof repairs with checked enclosing declarations and axiom audits. They

are not a transparent unchanged-input verifier optimization. Combining these repairs with historical accepted IDs is not a fresh full-population run or a pass-set preservation experiment.

The contrast with modern library proofs is negative. Seven matched Int/list certificate comparisons accept in both conditions, but all seven certificate variants are slower. Whole-source sums increase from 13.318 to 23.937 seconds; additional warm declaration checks increase from 2.328 to 5.758 seconds. The extra checks exclude original construction and first checking, so they do not isolate total kernel CPU. Further external literal/certificate tests recover no whole ProofNet, EPFL or APRIL source in the tested subsets. An intermediate tactic hit can still leave a later false or unproved obligation; closing that tactic is not acceptance of its enclosing source.

System-design implication. Selected deep reductions can benefit from a different certified algorithm, but that result does not generalize to every decision tactic or library. Certificate generation, correspondence proofs, first kernel checking, imports and complete-source acceptance must all be charged. The checked semantic boundary matters more than the size of a raw arithmetic loop. Existing external search/check architectures and native decision procedures remain prior techniques [8, 9].

The same boundary limits device conclusions. In the completed miniF2F counterexample-screening comparison, exact post-checking refutes intermediate goals in 29 ordinary-error candidates, removes no timeout and increases whole elapsed time by 8.86%. It is a small component with limited measured coverage, not a general GPU acceleration result. Failure of that implementation does not prove GPUs cannot help a different profiled workload. The external arithmetic oracle remains a frozen baseline, not a contribution or a new experimental direction.

9 What the characterization supports, and what remains unknown

The observations support six design priorities: declare build coverage; attribute system work before adding cores; isolate the import floor; separate failure frequency from tail concentration; measure observer perturbation; and evaluate proof-representation changes through complete native checking. They do not yet establish one generally superior verifier.

There are three major open evidence boundaries. First, all-source native cost estimation is incomplete. Completed samples, purposeful stresses and dependency work cannot be pooled into a whole-library point estimate, and cold preparation costs remain outside the compiler clocks. Second, generated-software availability and performance must be established independently of published task scaffolds and benchmark scores. Third, current operating-system counters are observational; causal attribution awaits the privileged profiling campaign. No missing result is filled with an estimated speedup.

The artifacts preserve unsuccessful prototypes, signal-limited captures, budget interruptions and uninvoked dependencies. An earlier 8,031-unit state-reuse plan is interrupted rather than complete: 2,640 original records are archived, one panic record has unknown elapsed time and at least 1,273 later observed ledger entries are currently unrecoverable after storage eviction. Those missing records are neither ordinary failures nor evidence of zero cache opportunity. The present paper uses only retained data with declared scope.

Public corpus selection also limits generalization. EPFL's construction filters some infrastructure failures, invalid outputs and missing declarations; APRIL mutation subsets are not natural arrival traffic. AutoVerus source-sorted replay is not a historical arrival trace. Repository buildability, per-file compiler exit, tactic success and axiom-clean acceptance remain different outcomes. **[RESULT: Representative software audit, complete profile tables and repeat uncertainty required for the final characterization.]**

10 Related work

Lean and Mathlib define the proof-construction/checking stages and dependencies being measured [3, 8]. Lean Kernel Arena compares post-export kernel implementations [14]; that boundary excludes source elaboration and import preparation. Our primary library audit instead tracks released source, build domains and native compiler work, without treating token counts as timed checks. Vero and Vericoding measure verified software generation [30, 2]. We use their artifacts only when evaluated outputs or generated sources are actually public, and distinguish their task-solving metrics from the cost of verification.

Persistent environments, snapshotting and warm servers already amortize preparation [7, 25, 11, 22]. Process copy-on-write is established [17]; ordinary resource scheduling is not a new algorithm here. Proof-level parallelism and futures precede this work [28, 15]. Our counters motivate a diagnosis of realized parallelism, not a claim to have invented parallel proof checking.

Fine-grained verification-result caching [16], tactic/prefix reuse [1, 26] and simplifier memoization [10] are relevant design alternatives. Dynamic dependency tracking [23, 5] does not make a lexical repeated tactic into a sound failure-cache hit. Likewise, algorithm selection and restart scheduling have established theories [29, 18, 24]. We characterize the work to which such techniques could apply; we do not claim a new cache or

scheduler from the measurements alone.

11 Conclusion

Verification of AI-generated formal artifacts has workload- and boundary-dependent costs. The research-library evidence exposes substantial system CPU, low realized core use, a repeated fresh-process floor and incomplete default-build coverage. Candidate streams expose different failure and tail structures; resource limits and profiling can change the observed outcome. Targeted certificate repair illustrates an algorithmic response to selected pathological reductions, while its negative transfer results limit that response's scope. [**RESULT: Final generated-software, import-attribution, perf-cause and population-cost findings.**] The characterization argues for measuring the complete acceptance path and its actual workload before selecting an acceleration mechanism.

References

[1] Lasse Blaauwbroek, Josef Urban, and Herman Geuvers. The tactician (extended version): A seamless, interactive tactic learner and prover for coq, 2020. Extended version of the CICM paper.

[2] Sergiu Bursuc, Theodore Ehrenborg, Shaowei Lin, et al. A benchmark for vericoding: formally verified program synthesis, 2025. Output tree inspected at 387cd69996792d452ead7b0460f36ee4c5cdd148.

[3] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction (CADE 28)*, pages 625–635, 2021.

[4] EPFL-LARA. Lean evaluation benchmark, 2026. Pinned release 0bbf31331bd1c726f539b311cce8b058a821f15c; accessed 30 October 2026.

[5] Matthew A. Hammer, Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. Adapton: Composable, demand-driven incremental computation. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 156–166, 2014.

[6] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying rust programs using linear ghost types. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1), 2023.

[7] Lean community. Lean community repl, 2026. Accessed 2026-10-07; year denotes access year.

[8] Lean community. Mathlib, 2026. Accessed 2026-10-07; year denotes access year.

[9] Lean contributors. Lean 4.9.0-rc1 ordinary decision tactic implementation. Pinned evalDecide handler; accessed 30 October 2026.

[10] Lean contributors. Lean4.22.0 release: simplifier cache reuse, 2025. Simp section; PR 8880.

[11] Lean contributors. Lean 4.32.0 release: incremental snapshots, 2026. Accessed 2026-10-07; year denotes access year.

[12] Lean contributors. Lean 4.34.1 auxiliary-lemma implementation, 2026. Branch-local cache lookup; accessed 30 October 2026.

[13] Lean contributors. Lean 4.34.1 decision-tactic implementation, 2026. doKernel invokes mkAuxLemma; accessed 30 October 2026.

[14] Lean contributors. Lean kernel arena, 2026. Accessed 2026-10-07; year denotes access year.

[15] Lean contributors. Lean.Elab.Parallel: parallel tactic-monad combinators, 2026. Source documentation; accessed 29 October 2026.

[16] K. Rustan M. Leino and Valentin Wüstholz. Fine-grained caching of verification results. In *Computer Aided Verification (CAV), Part I*, pages 380–397, 2015.

[17] Linux man-pages project. fork(2) linux manual page, 2026. Accessed 2026-10-07; year denotes access year.

[18] Michael Luby, Alistair Sinclair, and David Zuckerman. Optimal speedup of Las Vegas algorithms. *Information Processing Letters*, 47(4):173–180, 1993.

[19] Meta Platforms. Lean Kernel Verify: Compiler-based safety checker for gpu kernels, 2026. Pinned release 129e65bfd8efbf6c45710732d205c941ef65928c; accessed 30 October 2026.

[20] Microsoft Research and AutoVerus contributors. AutoVerus: Artifact evaluation, 2026. Source revision cbf9c0c6337b224fd8e5b7cb4e01ae65c0f98bc1; accessed 16 October 2026; year denotes access year.

[21] OpenAI math contributors. math: Mathematical manuscripts and supporting proof artifacts, 2026. Pinned source release 4eaee87548250b2682affda7d05d947c05554bae; accessed 30 October 2026.

[22] Project Numina. Kimina lean server, 2026. Accessed 2026-10-07; year denotes access year.

[23] Salsa contributors. The red-green algorithm, 2026. Official algorithm documentation; accessed 30 October 2026.

[24] Ziv Scully, Mor Harchol-Balter, and Alan Scheller-Wolf. SOAP: One clean analysis of all age-based scheduling policies. arXiv:1712.00790v2, 2018.

[25] Austin Shen and Yunong Shi. Keep the proof state live: Snapshotting for efficient tactic search in lean 4, 2026.

[26] SHRED contributors. SHRED: Reuse repeated work across thousands of Lean proof attempts, 2026. Software and experimental artifact; revision a6616e99534ce777dcebfd56566eaaffb63997a8; accessed 16 October 2026.

[27] UW Math AI. APRIL: Lean proof repair dataset, 2026. Pinned release 4e3075279b4aedfb28fe435d08a8caf76272586d; accessed 30 October 2026.

[28] Makarius Wenzel. Read-eval-print in parallel and asynchronous proof-checking. In *User Interfaces for Theorem Provers*, volume 118 of *EPTCS*, pages 57–71, 2013.

[29] Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32:565–606, 2008.

[30] Zhe Ye, Hantao Lou, Yuechun Sun, et al. Vero: Can ai agents build formally verified software repositories?, 2026. Official repository inspected at 0a7325df9e9e6dbc275c0ad483b3d1cbe38d9b09; evaluated run outputs not established.